

23/9/20
Let's build:

Prob 3: Analyst & Auditor.

[X. Agent design and Orchestration]

Solution should consist of 2 Agents *

- Analyst (researcher)
- Auditor (Evaluator)

Analyst: Part ①

- Should answer open research questions.
- Along with that provide evidence gathered to produce/generate the answer (live web *)
 - ~~Agent~~ should scrap the web to answer the question, not answer based on pre-trained knowledge

Constraints (kinda)

- must plan an approach. before searching ①
- Use tools (P.F & web scrapping)
- Run parallel (cross check various valid sources),
 - ② carry the info across the ques (helps to get an answer faster) *

- Answer carry Citation *
- Include memory.

(Test atleast with 3 ques by ↑ difficulty, should utilize the memory)

As of now → The approach can be:

① Model should understand what info needed from the input to answer

⇒ ② Check Memory & retrieve related info, if any related question asked previously.

change needed.

③ whole may be that would take time.

④ Parallely run many sources to decide which needed

⑤ Decide what newly has to be searched.

- ⑥ fetch pages & search ⑤ Cross - check
claims occur in parallelism ④... → ⑦ Answer
drop redundant sources / info. with citation.

Auditor: Part ①.

↳ Takes an Analyst answer and Verifies it.

Plan:

- ↳ For each claim → open the citation ⑤ Confirm
the source what claims with mark (S, U, C)
↳ Flags claims on no citation →

Need to be Done:

- ↳ For Every question → Full Trace Needed.
C plan, full calling, what
came, ③ (with actions).

(As Now Plan for 10 EPS Cost) →

- Plan the sources disagreement
→ Send feedback from auditor to analyst, so
next run be better →

(As for plan the Approach)

- ↳ it has to newly search for every question
even though it has info present in memory → it should act
as the primary citation & new search act as the 2nd source
⑤ Cross - verify →

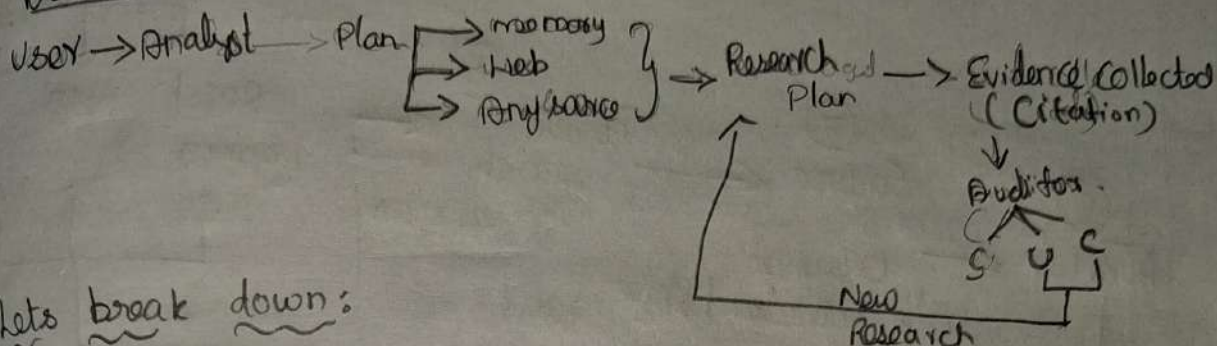
24/11

Now, Problem and requirement Understood!!

Before Implementation, check for existing Systems:

Our Main Target : Evidence based / driven Research System
+
Strict Audit.

Asked to:



Let's break down:

Before that:

Imp thing has to learn -> Adaptive Decomposition. Comes into Chunks.

~~Key~~

=> checked for Existing Analyst & Auditor Agent & Analyse the limitations of those existing Agents. Try not to commit the same.

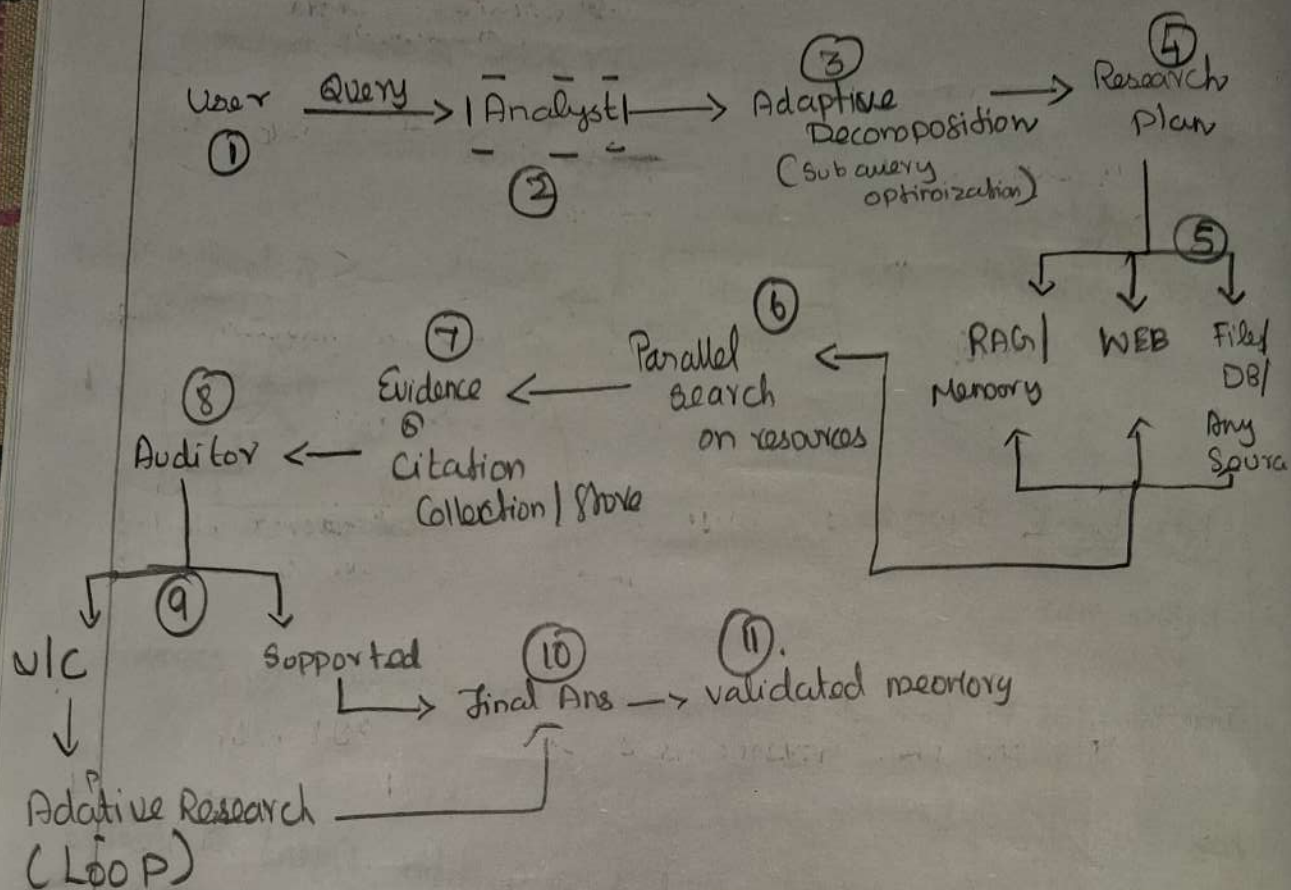
Live Web searching agent should involve: Checklist:

- > every decomposition
- > ReAct Framework
- > HTML to md Extraction & Scrubbing.
- > React
- > O. OPT
- > Parallelization
- > Data Cleaning
- > Cross Source Eval
- > planning & execution loops

Moreover, Scoreboard agent ^{AD}, Hopscript Agent, Tandy Search API, beautiful Soup 4.

Search or Data API : EXA Search / Tavily.
Or, Framework:
MCP

Before Research → My thought on architecture should be !!



(Should Add Tavily Search API → To retrieve cleaner factual retrieval)
 ↓
 Search Tool.

(After Page fetching → Parsing & extraction → Beautiful Soup
 it is a py library) *

⇒ So, Right after (5), In (6) i.e parallel search done with Tavily tool → URL(s)

↓
 Page fetched as HTML

HTML
 Parse & Content
 extract by
 BeautifulSoup

Clean Text ←
 Now, Pass it to
 (7) *

Let's look into the fundamentals of Agentic AI.

1. Model & Compute Layers



Foundational Models (LLM)
Compute & Infra (GPU, CPU)

2. Data & Memory Layer



Context Stores & Knowledge base
Data pipeline

3. Tooling & Integration Layer



Tool execution & Sand box
Protocols & interoperability

4. Orchestration Layer



Workflow Management (State tracking, planning)
Handlers & Runtime control

5. Security, Governance & observability layer



Guardrails & Compliance
Traceability & Auditing

6. Application Layer.

I Think, In the prev architecture, we forget about Interoperability issues, Security (Guardrail issues), Governance & Observability layer itself.

Refine the flow / Architecture:

Core flow be like: Analyst → Research → Evidence



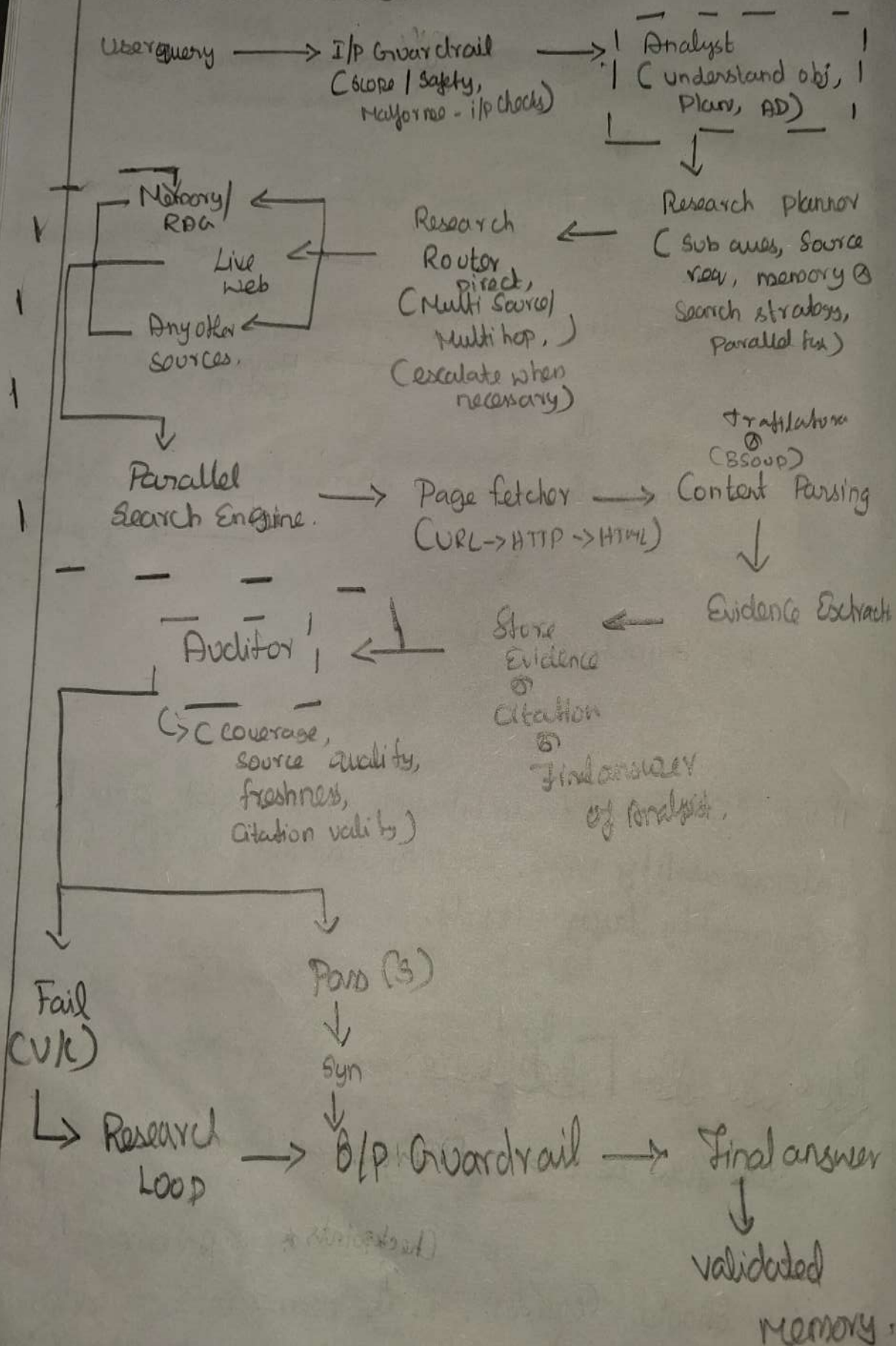
Checkpoints *

Answer ← Auditor

Anyway should consider: state management, deterministic orchestration (planning), guardrails, tool governance, observability,

bounded Execution, ~~Human~~ approval points & explicit failure handling. (retry, fallback, skip, reopen, stop)

Now the final Architecture:



But Here

Budget,
Configuration

Telemetry *

→ Think
4 sources

→ Audit

Let keep
has to be

(Orches
Tracing

25/6/26.

But Here, have to be cautious abt: State, CP,
Budget, Tracing, Timeout, Tool permission, Human approval,
Configuration, Retry, ... Loop Control ?? !:

Telemetry *

↓
Agent Harness

→ Think about Redundant info ?? What if All the
4 sources are same article from diff web-sites ??

→ cautious about Prompt-injection Defense *

Let keep the before architecture as final, Now technologies
has to be finalized *

(Orchestration, ^{Langgraph} LLM, Search tools, fetching, parsing, Memory,
Tracing, Evaluation)

25/6/26